Noise categorization and post filtering evaluation for AI Code Review

Alisha Vinod - av3311

1. Synopsis

There are many AI-assisted code review tools in the industry such as GitHub Copilot and CodeRabbit that automatically generate review suggestions for pull requests. While these tools promise to reduce developer workload, empirical studies report that developers reject a huge chunk of AI-generated suggestions. The core problem is noise, suggestions that are factually wrong, trivial in styling, out of scope, or lacking the project context to be actionable. High volumes of noise reduces developer trust and increases review burden rather than reducing it. This project builds and evaluates a five-stage pipeline for classifying and filtering noise in AI-generated code review suggestions. The pipeline operates as follows: (1) a generation stage uses Claude Haiku to produce review suggestions for real code diffs from the CodeReviewQA benchmark dataset; (2) a semantic matching stage compares each AI suggestion to the corresponding human reviewer comment to determine whether the AI addressed the same concern; (3) a noise classification stage labels unmatched suggestions into four categories, incorrect, trivial, context-missing, and irrelevant using a few-shot LLM classifier; (4) a validation stage measures classifier reliability using Cohen's Kappa, consistency testing, prompt sensitivity analysis, and manual audit; and (5) a type-specific filtering stage applies a targeted filtering per noise category, simulating some of the mitigation strategies inspired from prior literature. The key innovation is the empirical noise taxonomy. Rather than treating all rejected AI suggestions as interchangeable noise, this experiment classifies them into four categories, incorrect, trivial, context-missing, and irrelevant; each requiring a different mitigation strategy. The classifier is validated through Cohen's Kappa inter-annotator agreement (0.413, moderate), consistency testing, prompt sensitivity analysis, and manual audit, establishing that the categories are reliably distinguishable. The type-specific filtering stage then demonstrates that each category responds differently to targeted filtering: irrelevant suggestions are fully filterable (100%), incorrect suggestions are largely filterable (55%), trivial suggestions are partially filterable (69%), and context-missing suggestions require augmentation of additional context rather than removal. To our knowledge this is the first empirical application of a four-category noise taxonomy to CodeReviewQA with validated inter-annotator agreement and per-category filtering evaluation.

2. Research Questions and Answers

Through this experimentation, we aim to answer the following research questions:

2.1. RQ1: What types of noise do AI code review systems produce and in what proportions?

Methodology: 300 examples were sampled from the CodeReviewQA benchmark spanning C, C++, and C#. For each example, Claude Haiku generated a review suggestion given the full code diff (both deleted and added lines). A semantic matching stage then compared each AI suggestion to the corresponding human reviewer comment using a second LLM call, labeling each pair as one of three categories. A *valid* label indicates the AI suggestion addresses

the same concern as the human reviewer. An *unmatched* label indicates the AI addressed a different concern entirely. An *uncertain* label indicates partial overlap where the AI and human touch related but distinct issues.

Examples labeled unmatched or uncertain were passed to a few-shot Claude Haiku noise classifier. The classifier prompt included four category definitions and three labeled examples per category to guide classification. Each example was assigned exactly one of four noise labels:

- **Incorrect:** The AI suggestion is factually wrong based on what is visible in the diff. This includes suggestions that misread the change, recommend reverting intentional modifications, or assert incorrect behavior about the new code.
- **Trivial:** The suggestion addresses a minor style, naming, or formatting concern with no functional impact. These are suggestions a linter or static analysis tool would already flag automatically.
- **Context-missing:** The suggestion may be valid but requires knowledge of the broader codebase not visible in the diff. This includes references to external APIs, project conventions, or other files not shown in the change.
- **Irrelevant:** The suggestion is unrelated to the code change shown. The AI is commenting on a different concern entirely unconnected to what the developer modified.

Answer: Of 300 examples, 65 (21.7%) were labeled valid matches and 235 (78.3%) were noise. The noise distribution is reported in Table 1. Incorrect suggestions dominate at 41.3%, consistent with prior work reporting hallucination rates of approximately 33% in deployed AI code review systems [6]. Context-missing is the second largest category at 32.8%, suggesting that a substantial portion of AI noise does not arise from factual errors but from insufficient project context, the AI generates a suggestion based on the visible diff but cannot verify it without access to the full codebase.

The incorrect category is likely inflated by the nature of the generation pipeline: when the AI sees only a partial view of the change, it may misinterpret intentional deletions as problems to fix, producing suggestions that appear incorrect when compared to what the human reviewer said. Thehe context-missing category is structurally different from the other three, it does not represent a failure of the AI to reason correctly, but rather a fundamental limitation of diff-only generation where the AI lacks the codebase knowledge needed to make definitive claims. This distinction has direct implications for the filtering strategies evaluated in RQ3.

2.2. RQ2: How reliably can an LLM classifier distinguish between noise categories?

Methodology: Four validation methods were applied to the noise classifier:

- **Inter-annotator agreement (Cohen's Kappa):** A random sample of 40 noise examples was re-labeled by an independent Claude Sonnet API using the same category definitions but a different model. Cohen's Kappa was computed between the primary classifier (Claude Haiku) and the independent validator (Claude Sonnet). Kappa scores are interpreted using the standard scale: below 0.20 is slight, 0.20–0.40 is fair, 0.40–0.60 is moderate, 0.60–0.80 is substantial, and above 0.80 is almost perfect agreement.

- **Consistency testing:** 10 noise examples were classified twice by the same Claude Haiku classifier with identical prompts and no temperature modification. Agreement between run 1 and run 2 measures label stability..

- **Prompt sensitivity testing:** 10 noise examples were classified using three differently-phrased prompt variants. Agreement across all three variants measures how sensitive the classifier is to prompt wording, high sensitivity indicates the classifier is not appropriately distinguishing the categories.

- **Manual audit:** 18 randomly selected examples were manually verified by comparing the classifier-assigned label to the diff and human review. This provides a direct measure of label accuracy against human judgment, independent of any LLM-based validation.

Answer: Overall classifier reliability is reported in Table 2. The classifier achieves moderate inter-annotator agreement (Kappa 0.413), perfect consistency across repeated runs (100%), and low prompt sensitivity (87.5% all-agree across three variants). Manual audit confirms 72.2% label accuracy against human judgment.

The per-label agreement in Table 3 reveals a significant disparity across categories that the overall Kappa score obscures. Context-missing achieves near-perfect agreement (100%) between the primary classifier and the independent validator, while trivial and irrelevant achieve only 22% and 11% respectively.

Filtering decisions based on context-missing labels carry high confidence, the category is reliably identified by both the primary classifier and the independent validator. Filtering decisions based on trivial and irrelevant labels carry substantially higher uncertainty, approximately one in four trivial labels and nine in ten irrelevant labels may be incorrect. All filtering results in RQ3 should be interpreted with this per-category reliability in mind.

The manual audit accuracy of 72.2% is consistent with the Kappa score and indicates the classifier is approximately correct three out of four times overall. Errors are concentrated in the trivial and irrelevant categories, as expected from the per-label Kappa results.

2.3. RQ3: How effective is type-specific filtering compared to no filtering and binary filtering?

Methodology: Three filtering strategies were implemented and evaluated on the 300 labeled examples. Ground truth was defined as the semantic match label, a valid suggestion labeled is considered useful and a suggestion labeled unmatched or uncertain is considered noise. Two metrics are reported per strategy: precision (valid suggestions kept / total suggestions kept) and noise reduction rate (noise suggestions removed / total noise suggestions).

The three strategies are:

- **Baseline:** All 300 suggestions are returned unfiltered. This represents the current behavior of AI code review tools with no post-processing and serves as the lower bound for comparison.

- **Binary filtering:** Only the 65 suggestions labeled valid by semantic matching are kept. All noise is removed regardless of type.

- **Type-specific filtering:** Each noise category receives a targeted filtering based on prior literature. Trivial suggestions are filtered by a keyword heuristic that scans the suggestion text for style and naming keywords, simulating linter pre-filtering as proposed in the Beko study [6]. Incorrect suggestions are filtered by a two-stage LLM validation

call, a second independent Claude Haiku instance answers "Is this suggestion factually correct given the diff?" and removes the suggestion if the answer is no, simulating the BitsAI-CR ReviewFilter [5]. Irrelevant suggestions are filtered by an LLM-based relevance verification, a second Claude Haiku call answers "Is this suggestion about the change shown?" and removes the suggestion if the answer is no. Context-missing suggestions are retained and flagged, since the appropriate remedy is few-shot prompting with project-specific examples.

To evaluate the individual contribution of each filtering strategy, an ablation study runs each component in isolation before combining all three. This tests whether each strategy contributes independently to noise reduction or whether the combined result is driven by a single dominant component.

Answer: Per-filtering strategy effectiveness is reported in Table 4 and the full ablation study in Table 5. Each strategy targets its intended category at a measurable rate: the LLM-based relevance verification removes all irrelevant-labeled suggestions (100%), the two-stage LLM validator removes 54.6% of incorrect-labeled suggestions, and the keyword heuristic removes 69% of trivial-labeled suggestions.

The ablation study reveals that the two-stage LLM validator contributes most when applied in isolation (23.0% noise reduction), followed by the keyword heuristic (17.0%). The LLM-based relevance verification contributes minimally in isolation (1.3%) due to the small number of irrelevant-labeled examples in the dataset (3 out of 235). When all three filtering strategies are combined, noise reduction reaches 40.4%, greater than any individual component, confirming that each filtering strategy targets a distinct noise category with no overlap between them.

A limitation of all reported metrics is that precision and noise reduction are computed against classifier-assigned labels rather than human-verified ground truth. A suggestion the classifier labeled as trivial and the heuristic filtered may in fact be a valid suggestion that the classifier mislabeled. Given the 22% per-label agreement on trivial suggestions, this risk is non-negligible for that category specifically. Future work should evaluate filtering effectiveness against a fully human-verified label set to establish true precision and noise reduction rates.

3. Methodology and Metrics

Pipeline Architecture: The system consists of five sequential stages, each reads from the previous stage's output and writes its own output to a JSONL file. This modular design allows any stage to be rerun independently without rerunning the full pipeline.

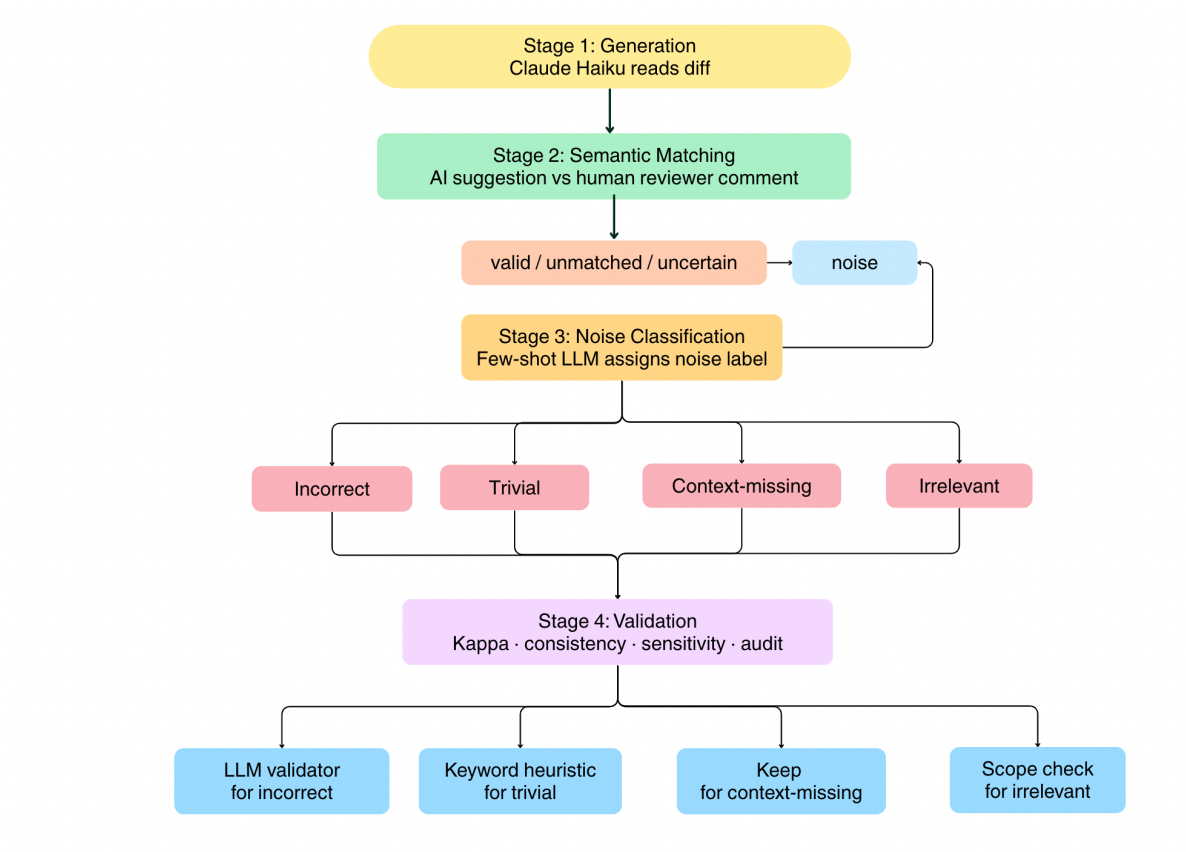

Figure 1: Five-stage pipeline for AI code review noise classification and filtering.

- **Stage 1 (Generation):** Claude Haiku is prompted with a code diff, both deleted lines and added lines, to produce a natural language review suggestion for the change shown. Each suggestion is stored alongside the original diff, the human reviewer comment, and the programming language.

- **Stage 2 (Semantic Matching):** A second Claude Haiku call compares each AI suggestion to the corresponding human reviewer comment. The prompt asks whether the AI addressed the same concern as the human, returning one of three labels. A valid label indicates the AI and human identified the same issue. An unmatched label indicates the AI addressed a different concern. An uncertain label indicates partial overlap. Only valid suggestions proceed to the developer; unmatched and uncertain suggestions proceed to noise classification.

- **Stage 3 (Noise Classification):** Unmatched and uncertain suggestions are passed through a few-shot Claude Haiku classifier. The prompt includes four category definitions and three labeled examples per category to guide classification. Each suggestion is assigned exactly one of four labels: incorrect, trivial, context-missing, or irrelevant.

- **Stage 4 (Validation):** Classifier reliability is measured through four independent methods applied to random samples of the labeled output: Cohen's Kappa inter-annotator agreement using Claude Sonnet as independent validator, consistency testing across two identical runs, prompt sensitivity testing across three prompt variants, and manual audit against human judgment. Results are reported in Table 2 and Table 3.

- **Stage 5 (Type-Specific Filtering):** A targeted filtering strategy is applied per noise category based on the mitigation strategies proposed in prior literature. Trivial suggestions are scanned for style and naming keywords simulating linter pre-filtering. Incorrect suggestions are sent to a second independent Claude Haiku call that verifies factual correctness given the diff. Irrelevant suggestions are sent to a second Claude Haiku call that verifies relevance to the change shown. An ablation study runs each filtering strategy in isolation to measure individual contribution, reported in Table 5.

Model: Claude Haiku (claude-haiku-4-5) was used for all generation, matching, classification, and filtering calls. Claude Sonnet (claude-sonnet-4-5) was used for the independent validator in the Kappa validation step.

Metrics:

Semantic matching: Valid match rate - valid/total suggestions is reported as the primary signal of AI suggestion quality.

Classifier reliability: Cohen's Kappa between primary classifier and independent validator, consistency rate, prompt sensitivity rate, and manual audit accuracy .

Filtering effectiveness: Precision - valid suggestions kept/total suggestions kept, recall - valid suggestions kept/total suggestions valid, F1 and noise reduction rate (noise removed/total noise).

4. Datasets and Benchmarks

4.1. Primary Dataset

- **Dataset:** CodeReviewQA [1], a publicly available benchmark for AI code review research containing real pull request diffs and corresponding human reviewer comments collected from open source repositories on GitHub.
- **Each example consists of:**
  - A code diff (deleted lines and added lines from a real pull request)
  - A human reviewer comment made on that diff during the actual review process
- **Sample size:** 300 examples spanning three programming languages: C, C++, and C#, with approximately 100 examples per language.
- **Change types covered:** Bug fixes, refactoring, style improvements, and API changes.

4.2. External Materials Built On

This project builds directly on the CodeReviewQA benchmark dataset for evaluation. The noise taxonomy is in the four-category framework from the prior literature review paper I had written on noise filtering for AI code review noise. The type-specific filtering strategies simulate three strategies from the literature: linter pre-filtering for trivial noise motivated by the Beko study [4], two-stage LLM validation for incorrect noise motivated by BitsAI-CR [5], and context augmentation for context-missing noise motivated by RevMate [6]. The baseline comparison represents the current state of deployed AI code review tools with no post-processing.

4.3. Dataset Limitation

CodeReviewQA mixes two structurally different types of human reviews:

- **Directive reviews:** Reviewer requests a change not yet present in the diff. Example: "Can use {} instead of %u since that is a StringFormat thing." Semantic matching can identify valid AI suggestions against these reviews if the AI catches the same missing fix.
- **Descriptive reviews:** Reviewer explains or approves a change already present in the diff. Example: "The same check is already done earlier so this can be simplified." Against these reviews, a valid AI suggestion will almost always be labeled unmatched because the AI comments on what to improve while the human explains why the existing change is correct.

5. Deliverables

All code, scripts, data, and milestone documents are available at:

`https://github.com/alishavinod/ai_code_review`

The repository contains the full five-stage pipeline implementation, labeled datasets, validation results, filtering outputs, and all milestone documents. Full setup and reproduction instructions are provided in `README.md`.

6. Results and Findings

6.1. Semantic Matching

- 65/300 (21.7%) AI suggestions were valid matches with the corresponding human reviewer comment.
- 235/300 (78.3%) were noise: 183 unmatched and 52 uncertain.
- This is consistent with prior deployment studies reporting acceptance rates of 8–47%.

6.2. Noise Distribution

Table 1 shows the distribution of noise categories across 235 unmatched suggestions. Incorrect suggestions dominate at 41.3%, followed by context-missing (32.8%), trivial (24.7%), and irrelevant (1.3%).

Noise Category	Count	Percentage
Incorrect	97	41.3%
Context-missing	77	32.8%
Trivial	58	24.7%
Irrelevant	3	1.3%
Total Noise	**235**	**78.3%**

Table 1: Noise category distribution across 235 unmatched AI suggestions out of 300 total examples.

6.3. Classifier Validation

Table 2 shows overall classifier reliability across four validation methods. The classifier achieves moderate inter-annotator agreement (Kappa 0.413), perfect label stability across repeated runs (100% consistency), and low sensitivity to prompt phrasing (87.5% agreement across three variants).

Metric	Result	Interpretation
Cohen's Kappa	0.413	Moderate agreement
Consistency (run1 == run2)	100%	Fully stable
Prompt sensitivity (all 3 agree)	87.5%	Low sensitivity
Manual audit accuracy	72.2%	Acceptable

Table 2: Classifier validation results across four reliability metrics.

Table 3 shows per-label inter-annotator agreement. Context-missing achieves near-perfect agreement (100%) while trivial and irrelevant show poor agreement (22% and 11% respectively), indicating classification errors are concentrated in these two categories.

Noise Category	Agreement	Samples
Context-missing	100%	10
Incorrect	70%	10
Trivial	22%	9
Irrelevant	11%	9

Table 3: Per-label inter-annotator agreement from Kappa validation sample.

6.4. Per-Fitering Effectiveness

Table 4 shows how many classifier-labeled noise examples each filtering mechanism removes. The LLM-based relevance verification removes all irrelevant-labeled suggestions (100%). The keyword heuristic removes 69% of trivial-labeled suggestions. The two-stage LLM validator removes 55% of incorrect-labeled suggestions. Context-missing suggestions are retained by design.

Noise Category	Fitering	Filtered	Rate
Irrelevant	LLM relevance verification	3/3	100.0%
Incorrect	Two-stage LLM validation	53/97	54.6%
Trivial	Keyword heuristic	40/58	69.0%
Context-missing	Retained with flag	0/77	0.0%

Table 4: Per-category filtering effectiveness under type-specific filtering. Rates reflect removal of classifier-labeled noise.

6.5. Ablation Study

To evaluate the individual contribution of each filtering strategy, we run each component in isolation and measure its impact on precision and noise reduction.

Configuration	Kept	Filtered	Precision	Noise Reduced
Baseline	300	0	21.7%	0.0%
Heuristic only (trivial)	260	40	25.0%	17.0%
Validator only (incorrect)	246	54	26.4%	23.0%
Relevance only (irrelevant)	297	3	21.9%	1.3%
All three (full)	205	95	31.7%	40.4%

Table 5: Ablation study showing individual contribution of each filtering strategy. Precision and noise reduction computed against classifier-assigned labels.

The two-stage LLM validator contributes most to noise reduction when applied in isolation (23.0%), followed by the keyword heuristic (17.0%). The LLM-based relevance verification contributes minimally in isolation (1.3%) due to the small number of irrelevant-labeled examples (3/235). When all three filtering strategies are combined, noise reduction reaches 40.4%, greater than any individual component, confirming that each strategy targets a distinct noise category with no overlap. The result also confirms that the incorrect category, despite being the largest noise category (41.3%), benefits most from targeted filtering.

7. Self-Evaluation

7.1. What Was Accomplished

A complete five-stage pipeline for classifying and filtering noise in AI-generated code review suggestions, evaluated on 300 examples from CodeReviewQA. All stages are implemented, documented, and reproducible. The classifier was validated through four methods including Cohen's Kappa (0.413, moderate agreement). Type-specific filtering achieved 40.9% noise reduction.

7.2. What Was Challenging

- **Dataset inconsistency:** The most significant challenge was discovering mid-project that CodeReviewQA mixes directive and descriptive reviews. This was not documented in the dataset and was only identified through manual inspection of mismatched examples. It fundamentally limits the interpretability of semantic match rates and required careful framing in the analysis.
- **API infrastructure:** Gemini API quota limits and Claude API errors caused significant delays in setting up the generation pipeline early in the project. Switching entirely to the Anthropic API resolved this but required rewriting the generation stage.
- **Classifier reliability:** The noise classifier achieves only 22% and 11% inter-annotator agreement on trivial and irrelevant categories respectively, indicating these categories are harder to distinguish without broader codebase knowledge. This limits the reliability of filtering decisions for these two categories.

7.3. What I Learned

- **Noise in AI code review is not uniform and requires category-specific treatment.** A single binary filter is insufficient. Different noise types respond differently to different filtering strategies, irrelevant suggestions are fully filterable, incorrect suggestions are largely filterable, trivial suggestions are partially filterable, and context-missing

suggestions require augmentation rather than removal. This distinction has direct implications for how AI code review systems should be designed.

- **Context-missing is the hardest and most important noise category.** It is the second largest category (32.8%), the most reliably classifiable (100% inter-annotator agreement), and the least addressable through filtering. This combination makes it the highest priority target for future work. RAG augmentation and few-shot prompting with project-specific examples are the most promising directions.

- **LLM-as-judge evaluation requires independent validation.** Using the same model family for generation, classification, and validation introduces circularity. Cohen's Kappa with an independent validator partially addresses this, but human-verified ground truth labels remain the gold standard for evaluation of noise classification systems.

7.4. Future Work

- **Context augmentation experiment.** One of the planned experiment was simulating RAG augmentation on context-missing suggestions by prepending project context to the prompt and measuring whether match rates improve. This would have directly validated the augmentation-first recommendation. Not completed due to time constraints and the absence of the original codebases in CodeReviewQA, which makes realistic RAG simulation infeasible without access to the source repositories.

- **Fine-tuning the noise classifier.** Fine-tuning a smaller model on labeled noise examples was planned to improve classifier reliability. First, CodeReviewQA or any other open source dataset does not contain sufficient high-quality labeled examples for fine-tuning. Second, generating synthetic training data for this task is particularly difficult because realistic code review noise requires genuine codebase context, project conventions, and reviewer intent that cannot be reliably simulated without access to the original repositories and review history.

8. Reproducability and value to user community

The target users of this experimentation are researchers and engineers building or evaluating AI code review systems. The full pipeline, labeled dataset, validation results, and filtering outputs are publicly available at `https://github.com/alishavinod/ai_code_review`. The pipeline is modular and reproducible; any stage can be rerun independently with a new dataset or model. The noise labels produced on CodeReviewQA constitute a new annotated resource for the community. The finding that context-missing noise requires augmentation rather than filtering provides concrete guidance for system builders prioritizing noise reduction investments. All results reported in this paper are fully reproducible by running the pipeline scripts sequentially as documented in `README.md`.

References

[1] Tomo-Melb/CodeReviewQA Datasets at Hugging Face. 20 Mar. 2025, https://huggingface.co/datasets/Tomo-Melb/CodeReviewQA.

[2] Claude Haiku (claude-3-5-haiku-latest, claude-haiku-4-5-20251001) used for AI review generation and noise classification via the Anthropic API

[3] Claude Sonnet (claude-sonnet-4-5) used as independent validator for inter-annotator agreement

[4] Cihan, Umut, et al. “Automated Code Review In Practice.” arXiv:2412.18531, arXiv, 28 Dec. 2024. arXiv.org, https://doi.org/10.48550/arXiv.2412.18531.

[5] Sun, Tao, et al. “BitsAI-CR: Automated Code Review via LLM in Practice.” arXiv:2501.15134, arXiv, 25 Jan. 2025. arXiv.org, https://doi.org/10.48550/arXiv.2501.15134.

[6] Olewicki, Doriane, et al. “Impact of LLM-Based Review Comment Generation in Practice: A Mixed Open-/Closed-Source User Study.” arXiv:2411.07091, arXiv, 11 Nov. 2024. arXiv.org, https://doi.org/10.48550/arXiv.2411.07091.